

Commander Wars Custom AI — Design Notes (Consolidated)

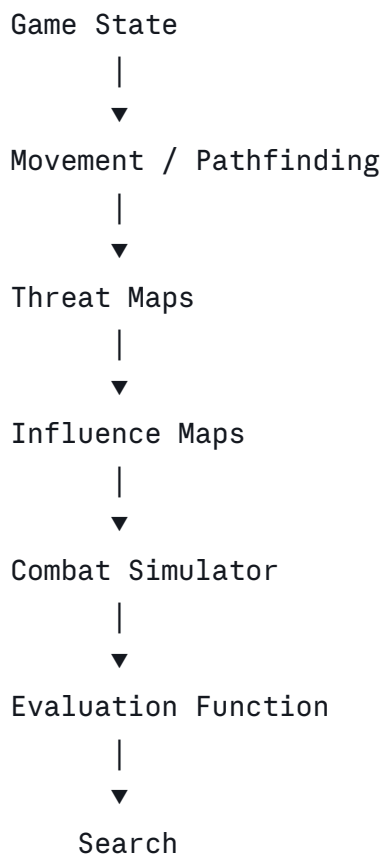
This file merges the four original planning docs with the suggestions discussed while reviewing them, so everything lives in one place.

Table of Contents

1. AI Architecture
 2. Threat Map
 3. Influence Map
 4. Evaluation Function
 5. Search Strategy
-

1. AI Architecture

Original plan



Suggested revision: two tiers, not one linear pipeline

The above reads as strictly linear, but Combat Simulator → Evaluation need to run **per candidate move**, not once per turn. Restructure as:

- **Once per turn (expensive, shared, describes the board):** Threat Maps, Influence Maps.
- **Once per candidate move (cheap, repeated many times, describes a hypothetical):** Combat Simulator → Evaluation, run against *virtual* state (see `virtualHp` pattern in Commander Wars' `Unit` class — a scratch overlay for hypothetical HP that avoids mutating real state or needing rollback logic, only an explicit reset once a candidate has been scored).

Open question to resolve before coding: does "Search" at the bottom mean literal lookahead (expectiminimax against a predicted enemy reply — see Section 5), or just "score every candidate move and take the max"? This changes how cheap Evaluation needs to be, since a real search calls it far more often.

2. Threat Map

Original plan

The threat map answers one question: **"What can the enemy do?"** It should not decide whether something is good or bad — that's the evaluation's job.

Should include:

- Direct attack threat (tanks, infantry, recons — which tiles can be attacked next turn)
- Indirect attack threat (artillery, rocket, battleship, missiles — needs movement + firing range)
- Capture threats (not just "can this infantry attack me" but "can it capture my HQ/airport/factory/city")
- Production threats (enemy threatens factory/airport/harbor)
- Property denial (enemy threatens cities you're capturing / your income)
- Counterattack opportunities (for every threatened tile, can the attacker itself be destroyed?)
- Multiple attackers (track *which* units threaten a tile, not just a boolean)
- Threat severity — store more than a boolean per tile:

Tile → Attackers → Expected Damage → Expected Cost Lost

Open questions from original doc:

- Enemy/friendly attack range → `CoreAI::getAttackTargets()`, `CoreAi::hasTargets()`
- Indirect fire handling — unresolved
- Fog interaction — unresolved
- Should threat include movement? (e.g. Tank: moves 6, attacks 1 → threat radius 7)
- Indirect threat isn't a circle — it's move-range *then* attack-range, so it's an annulus, not a disc
- Should threat account for HP / ammo / fuel?

Suggested additions

- **The "should not decide good/bad" boundary is the most important line in this doc — and the easiest to accidentally violate.** Even the shipped `NormalAi::calculateCounterDamage` blurs it: it mixes raw threat computation with value judgments (a `m_notAttackableDamage` threshold, a "split attention between multiple targets" discount). Gut-check while implementing: *"am I describing enemy capability, or am I deciding how much I care?"* — thresholds and multipliers that reflect your own priorities belong in Evaluation, not Threat.
 - **Resolve the open Q&A rows before coding** — indirect fire and fog handling in particular change the shape of the threat computation, not just its inputs.
 - **Use a real struct for per-tile threat data**, not an ad-hoc repurposed geometry type. (The shipped code reuses a `QRectF` as a float container via `.x()` — don't copy that; it's a source of confusion, not a shortcut.)
-

3. Influence Map

Original plan

Influence differs from threat: Threat = "what can happen immediately?" Influence = "which side controls this area?"

- **Friendly / Enemy / Net Influence** — Net = Friendly – Enemy. Positive = safe, negative = enemy-controlled.
- **Economic Influence** — cities, factories, HQ, airport, harbor.
- **Reinforcement Influence** — how quickly each side can reinforce an area; matters a lot on large maps.
- **Front line detection** — where friendly and enemy influence meet; useful for artillery, tanks, pushing, retreating.
- **Strategic terrain** — bridges, mountain passes, narrow roads deserve extra influence weight.
- **Vision influence (fog)** — recons, mountains, forests, hidden units.
- **CO-specific influence (long-term idea)** — influence map stays generic, weights shift per CO (e.g. Sami → infantry influence matters more; Grit → indirect influence matters more; Max → direct combat influence matters more).

Open questions from original doc:

- How should influence decay — by distance? Linear or exponential?
- Should influence depend on HP / cost / CO?
- Multiple separate maps (friendly, enemy, city, production)?
- Recompute per unit, per turn, or how often?

Suggested answers to the open questions

- **Decay by movement cost, not raw tile distance.** AW terrain isn't uniform-cost, so distance decay misrepresents projected power (a mountain range blocks influence far more than 3 empty tiles). Reuse existing pathfinding infrastructure for this rather than building separate distance decay (the shipped AI already builds a per-unit pathfinder object for its own threat evaluation — the same object is reusable here).
- **Start linear, not exponential.** Get a working v1 first; revisit the decay curve shape once you can watch the AI play and see where influence mis-predicts front lines.
- **Yes, weight by HP/cost.** A 2-HP tank shouldn't project the same influence as a full-HP one. Starting point: $\text{cost} \times (\text{hp} / \text{maxHp})$ as the "strength" a unit radiates outward.
- **Recompute once per turn**, not per unit — matches how the shipped AI caches its enemy data once per turn rather than recomputing per candidate move.

4. Evaluation Function

Core design principle (from original doc)

A move isn't good because it captures a city, blocks a factory, or damages a unit in isolation. It's good because it changes what each player can do over the next several turns.

Evaluate opportunities and constraints created, not just static board facts. Avoid double-counting the same strategic fact across subsystems — keep each subsystem focused:

- **Threat map** → reports possibilities ("this factory can be blocked")
- **Influence map** → reports territorial control ("this area is under enemy pressure")
- **Evaluation** → assigns value ("blocking this factory is worth +X because it denies production")

Categories (original "Best Version")

Material

- + Unit strength (cost × HP)
- + Property ownership / income
- + Production capability

Position

- + Terrain advantage
- + Attack coverage / mutual support
- + Defensive support / screening
- + Chokepoint control
- + Formation cohesion
- + Vision control

Tactical (comes almost entirely from the combat simulator)

- + Favorable trades, safe attacks, protected high-value units
- Expected retaliation, material loss, unsafe exchanges

Economic

- + Capture progress, future income, efficient repairs
- Repair costs, losing economy

Strategic Pressure

- + Property/production denial, factory/airport blocking
- + Forced enemy responses, initiative, capture contesting
- + Strategic diversion

Victory (should dwarf almost everything else)

- + HQ capture threats, elimination threats
- HQ danger, critical production loss

Expanded sub-concepts include: property/capture value, terrain advantage, attack coverage, defensive screening, expected retaliation (net exchange, not just immediate damage), unit exposure, threatened units, safe aggression, front line quality, chokepoint control, vision control, economy, and strategic objectives (HQ capture belongs in evaluation, not in the threat map — the threat map just reports the *possibility*).

Strategic Pressure / Denial Value — the "how much value is denied while this unit survives?" framing

Key sub-ideas from the original doc:

1. **Factory blocking** — value $\approx$ production denied $\times$ expected turns blocked.
2. **Income denial** — delaying a capture delays the income stream, which is real economic value even without owning the property yourself.
3. **Opportunity cost** — a seemingly bad trade (4000-cost Recon vs. 1000-cost Infantry) can be correct once you count delayed capture + denied income + forced response + tempo.
4. **Forced responses** — a move whose main purpose is "you have to answer this" consumes enemy production, movement, attention, and flexibility.
5. **Resource drain** — a damaged unit that survives still drains the opponent's repair budget every turn.
6. **Production efficiency** — high income with too few factories can't actually be converted into army strength; ask "who can convert money into military strength," not just "who has more money."
7. **Tempo** — who is dictating the game; emerges from forced responses, initiative, denial, mobility, pressure combined.
8. **Long-term property value** — a property is worth $\frac{\text{income} \times \text{expected remaining turns held}}{1}$, which is why the same capture is worth much more early game than late game.
9. **Strategic diversion** — a move that draws enemy production/attention to one front weakens their response elsewhere, even though nothing happened there.

CO-specific evaluation (explicitly deferred — long-term idea, not v1)

Shift evaluation *weights* per CO rather than changing the search itself (e.g. Sami: infantry/captures/economy early, survival/positioning late; Grit: indirect fire/terrain/vision; Max: direct combat/tank trades; Kanbei: preservation more valuable since units cost more; Eagle: air superiority more valuable).

Earlier, flatter evaluation function (kept for reference — easier to reason about directly)

- + Unit strength
- + Property income
- + Capture progress
- + Terrain advantage
- + Attack coverage
- + Defensive support / Screening
- + Expected retaliation
- + Vision control
- + Chokepoint control
- + Front line quality
- + Tempo
- + Production capability

- Exposed expensive units
- Threatened units
- Unsafe attacks
- Losing economy
- Poor positioning
- Isolated units

Suggested answer to "how does this become code?"

However elaborate the category structure, the runtime output has to collapse to a single comparable number per candidate — that's the only way to rank candidates against each other. The categories aren't a runtime data structure; they're an organizational tool for reasoning about the code and avoiding double-counting. Concretely, this is a **weighted linear sum of features** (same pattern as classical chess engine evaluation — material + piece-square tables + mobility + king safety, all weighted and summed into one number):

```
score = w_material * material_subtotal
      + w_position * position_subtotal
      + w_tactical * tactical_subtotal
      + w_economic * economic_subtotal
      + w_pressure * pressure_subtotal
      + victory_term (clamped, see below)
```

where each subtotal is itself a sum of smaller weighted terms (e.g. $\text{unit_strength} = \sum \text{cost}(u) \times \text{hp_fraction}(u)$ over owned units).

Three practical suggestions:

1. **Pick one common unit of measure.** The categories mix funds, tiles, turns, and vague "value." Force everything into the same currency — Advance Wars naturally offers cost/funds for this. The shipped code does exactly this: it converts raw HP damage into `fundsDamage` before accumulating a score. Do the same for positional/tempo/denial terms (even a rough funds-equivalent estimate keeps weights on a shared scale instead of being unitless magic numbers).
2. **Clamp win/loss conditions instead of just weighting them heavily.** A losing-the-HQ-next-turn state should score something like `-1,000,000` outright, not `-500 × a big multiplier` — clamping guarantees it can never be outvoted by an accumulation of smaller positive terms, whereas a purely weighted sum can be talked into ignoring an existential threat if weights are even slightly off.
3. **Build incrementally and keep a debug breakdown even after summing.** Start with just Material + Tactical, confirm sane play, then add one category at a time. Keep per-category subtotals available for debugging even though the runtime number is a single float — the shipped AI does this (`AI_CONSOLE_PRINT` logs `total score=...`
`counter damage=...` `influence damage=...` `building damage=...`) alongside the summed score). You'll want this the first time the AI does something inexplicable with forty scoring terms in play.

Suggested v1 scope: treat the flatter list as the actual first implementation target, and the categorized "Best Version" as the roadmap to grow into once v1 plays reasonably.

5. Search Strategy

Target approach: **expectiminimax** — chosen because Advance Wars damage rolls are genuinely stochastic (when luck isn't fixed/average), so a chance node averaging over damage-roll outcomes is the technically correct model, not just a nice-to-have. Branching is expected to explode faster than in chess because far more units move per turn than pieces move per chess ply.

Simplification re: luck nodes

The shipped AI itself dodges full luck branching — it calls its damage calculator with `GameEnums::LuckDamageMode_Average`, i.e. it uses expected-value damage as a single deterministic number rather than branching over the full luck distribution. This is a reasonable simplification to adopt: a true chance node over the full luck distribution multiplies an already-exploding branching factor by however many discrete damage outcomes are modeled, for comparatively little strategic benefit (average damage is usually a good enough proxy unless deliberately reasoning about high-variance all-or-nothing plays).

The bigger problem: what counts as "one ply"

In chess, a ply is "pick one piece, pick one legal move for it." In Advance Wars, a "turn" is "pick an action for every unit you own," and those per-unit choices aren't independent (moving unit A can block/unblock a tile unit B wants, or set up a combo attack). Treating "a full turn" as one search node gives a first-ply branching factor of roughly $\frac{\text{moves per unit}}{\text{unit}}^{\text{(number of units)}}$ — computationally hopeless even before reaching the opponent's reply node.

Practical fix: decompose a turn into a sequence of single-unit decisions rather than searching full turns as atomic nodes:

Turn = unit1's action → unit2's action → ... → unitN's action → end turn

Each individual unit action becomes its own node, evaluated against the intermediate hypothetical state (this is exactly where the `virtualHp`-style scratch-state pattern earns its keep — chaining many hypothetical sub-turns without touching real game state). Growth across N units is still combinatorial, but now boundable via **candidate pruning per unit**: use threat/influence maps to generate a short list of "interesting" tiles/targets per unit (attack candidates, capture candidates, retreat-to-safety candidates, support positioning) instead of enumerating every tile in movement range. This is the single biggest lever for taming branching factor.

Scoping suggestions for a first-generation AI

1. **Consider whether v1 needs real search at all.** The shipped `NormalAi` code doesn't appear to do multi-ply lookahead in the expectiminimax sense — it scores *this turn's* candidate moves against the *current* threat picture, with no explicit search into future turns beyond "what can immediately counter-attack me here." That's a one-ply greedy evaluator, not a searcher. Suggested v1 scope: get greedy per-unit best-move selection (evaluate every candidate for a unit, take the max — no tree) working and playing sanely first, and treat expectiminimax as a v2 upgrade layered on top once the evaluation function itself is trustworthy. A search is only as good as the evaluation at its leaves — debugging a bad evaluation function *and* a bad search tree simultaneously is much harder than debugging them one at a time.
2. **Depth 1 (your turn) plus a shallow, cheap predicted opponent reply is probably the realistic ceiling**, not deep lookahead. The predicted enemy reply doesn't need full expectiminimax on their side either — reuse the same greedy candidate-evaluation machinery used for your own move selection to produce a plausible enemy response. That gets the "am I walking into a trap" benefit of minimax without a symmetric full search on both sides.
3. **Alpha-beta doesn't transfer cleanly to chance nodes**, but a known analog exists for later: *-pruning (star1/star2), used in backgammon-style expectiminimax engines. Not a v1 concern — just worth knowing the name exists rather than reinventing it later.